Báo cáo kết quả huấn luyện + demo

Đề tài: Phát hiện trùng lặp code

Ngày báo cáo: 06/10/2026

Tổng quan

- Bài toán: phát hiện trùng lặp code trên tập dữ liệu POJ-104.
- Nội dung: kết quả huấn luyện 5 phương pháp (B1, M1–M4) + demo quét file `data.zip` (10 file C++, 45 cặp) + giải thích highlight trong modal.

Dữ liệu

- 51.991 hàm C++, 66.000 cặp, chia ba phần không trùng cặp.
- Ngưỡng quyết định chọn trên validation, giữ nguyên khi test.

Phần	Số cặp	Cặp dương	Dùng để
Train	45.000	22.500	huấn luyện
Validation	10.000	5.000	chọn ngưỡng quyết định
Test	11.000	5.500	báo cáo kết quả cuối

Các phương pháp

- **B1 MOSS** (baseline). Schleimer, Wilkerson & Aiken 2003. Gọi thẳng service thật `moss.stanford.edu` qua mosspy (EXP-014), không huấn luyện. Bản winnowing all-pairs chạy local (EXP-013) giữ làm tham chiếu và làm fallback cho demo khi chưa có tài khoản MOSS.
- **M1 CodeBERT**. `microsoft/codebert-base`. Ghép ba vector (A, B, độ lệch tuyệt đối A với B) rồi qua một lớp MLP.
- **M2 GraphCodeBERT**. `microsoft/graphcodebert-base`. Như M1, thêm quan hệ token qua cây cú pháp và luồng dữ liệu. Giữ nguyên siêu tham số M1.
- **M3 ASTNN**. Câu cú pháp rút gọn thành đồ thị câu lệnh, qua mạng đệ quy và mạng hai chiều. Huấn luyện từ đầu.
- **M4 CPG-GNN**. Đồ thị thuộc tính code property graph (6 loại cạnh: AST, CFG, ARGUMENT, TARG, PARAMETER-OUT, EVAL-ORDER), qua mạng GAT 3 tầng (4 head, hidden 128), đầu MLP như M1/M2/M3. Huấn luyện từ đầu (EXP-005).

Cấu hình huấn luyện

- Kaggle GPU Tesla T4, seed 42. Số epoch: CodeBERT 3, GraphCodeBERT 3, ASTNN 7, CPG-GNN 5.
- MOSS: không huấn luyện.

Cấu hình	CodeBERT	GraphCodeBERT	ASTNN	CPG-GNN
Mô hình có sẵn	codebert-base	graphcodebert-base	không, từ đầu	không, từ đầu
Độ dài đầu vào	256 token	256 token	không cắt	tối đa 1500 node
Learning rate	2e-5	2e-5	1e-3	1e-3
Batch size	16	16	32	16
Số tham số	126.416.641	126.416.641	428.001	203.649
Thời gian	7,0 giờ	6,9 giờ	40 phút	57 phút
Ngưỡng (val F1)	0,500	0,512	0,557	0,420

Quá trình huấn luyện

Phương pháp	1	2	3	Thời gian	Đánh giá
CodeBERT	**0,9191**	**0,9191**	0,9116	7,0 h	đã quá tay
GraphCodeBERT	**0,9155**	0,9138	0,9108	6,9 h	đã quá tay
ASTNN	0,7890	0,8141	0,8414	40 phút	tốt nhất **0,8530** ở epoch 4; dừng epoch 7
CPG-GNN	0,7798	0,7955	0,8004	57 phút	tốt nhất **0,8194** ở epoch 5; còn tăng

Giá trị là F1 validation từng epoch. CPG-GNN hội tụ chậm hơn hai transformer nhưng nhanh hơn ASTNN về số epoch đầu; val F1 còn tăng ở epoch cuối nên có thể train thêm nếu cần.

Kết quả

- Tập test, 11.000 cặp.
- Dấu ±: độ lệch chuẩn, bốc ngẫu nhiên lại tập test 2.000 lần.

Phương pháp	Precision	Recall	F1	ROC-AUC	Cặp đoán nhầm
B1 MOSS	1,000	0,002	0,003 ± 0,001	—	5.491
M1 CodeBERT	0,931	0,890	0,910 ± 0,003	0,973	363
M2 GraphCodeBERT	0,883	0,933	0,907 ± 0,003	0,971	682
M3 ASTNN	0,760	0,894	0,821 ± 0,004	0,889	2.138
M4 CPG-GNN	0,695	0,885	0,779 ± 0,004	0,864	2.768

Chi tiết M4 (EXP-005, `results/exp005_cpg_gnn_poj104/metrics.json`): validation P 0,780 / R 0,906 / F1 0,839 / ROC-AUC 0,923 / accuracy 0,826; test P 0,695 / R 0,885 / F1 0,779 / ROC-AUC 0,864 / accuracy 0,748, ngưỡng 0,420. M4 vượt baseline M1 về F1 nhưng kém ASTNN 4,2 điểm và kém hai transformer khoảng 13 điểm. Điểm mạnh của M4 là recall cao (0,885, chỉ sau M2/M3): bỏ sót ít clone, phù hợp làm vòng lọc đầu; điểm yếu là precision thấp (0,695): báo nhầm nhiều.

B1 không có ROC-AUC dù `---`: MOSS service chỉ trả top-`N` cặp khả nghi (EXP-014 chạy `show=1000`), không chấm điểm mọi cặp nên không xếp hạng được toàn bộ test. Số thật: trên test chỉ phát hiện 9/5.500 cặp dương (F1 0,003), và cả 11.000 cặp mang điểm 0 trừ số ít cặp trong report. Đây là giới hạn của **chính server MOSS** chứ không phải cài đặt: các cặp dương trong POJ-104 phần lớn là clone Type-3/4 (đổi tên biến, đảo câu lệnh), điểm winnowing chung thấp nên không lọt top-1000 toàn bộ tập. Bản all-pairs chạy local với cùng thuật toán (EXP-013) đạt F1 0,682 là minh chứng: MOSS giỏi khi chấm được *mọi* cặp, còn trong cách dùng "khoe report top-`N`" chuẩn như một công cụ đạo văn thì nó chỉ chặn được clone gần trùng hoàn toàn. So sánh công bằng nhất vẫn là EXP-013 (minh họa bên dưới), và với mốc đó các mô hình neural vẫn vượt rõ.

Demo quét file data.zip

`src/demo/data/data.zip`: 10 file C++ cùng bài `spit0035`, 45 cặp, chấm bằng 5 model qua `DemoService.analyze`. B1 gọi MOSS service khi ứng dụng có `MOSS_USER_ID` (hiện kèm cả đường dẫn report MOSS), ngược lại fallback về winnowing all-pairs local k=20, w=5 (EXP-013) — số bên dưới là lần chạy mặc định không có tài khoản nên dùng fallback. Ba nhóm:

- 3 cặp giống hệt (MOSS 1,000; M1/M2 1,000; M3 0,985; M4 0,962): cả 5 model clone, căn chỉnh phủ 100% hai phía.
- Cặp chéo 355523/355172: MOSS 0,333 nhưng 4 model neural đều clone — đúng ca Type-3/4 mà baseline bỏ sót.
- Cặp chéo 355179 với nhóm trên: MOSS 0,474 — vùng xám, cần model phân xử.

Cách đọc highlight trong modal Cặp trùng khớp

Chỉ một lớp màu:

- **Vàng = chỗ model đang chú ý** (evidence), đổi theo model ở cột trái (bảng dưới). Vàng ở đâu nghĩa là model dựa vào đó để cho điểm.
- **Không màu = phần còn lại** (khớp thuần hoặc khác nhau), không phải lỗi.

Model	Kỹ thuật	Đọc vệt vàng thế nào
B1 MOSS	k-gram chung	Chỗ nào trùng ký tự thì vàng ở đó; gần với "khớp" nhất. Không có k-gram chung thì không có vàng.
M1/M2	gradient × input (saliency)	Token nào làm đổi điểm mạnh nhất thì vàng, tính riêng từng phía. Hai phía vàng khác chỗ nhau là bình thường.
M3 ASTNN	cosine vector RvNN	Cặp câu lệnh giống nhau nhất lẫn nhau (hai chiều đều chọn nhau) thì vàng. Bỏ gốc bao cả hàm.
M4 CPG-GNN	attention GAT tầng cuối	Node nào được chú ý trên trung bình thì vàng, map về dòng qua trường line của exporter. Là "GAT nhìn đâu", không phải "đoạn nào khớp".

Cặp 355523/355172 (chạy 06/10/2026): B1 15 k-gram chung; M1 63 span; M2 62 span; M3 10 cặp câu lệnh; M4 7 span GAT. Năm vệt vàng khác nhau là đúng thiết kế — mỗi họ model "nhìn" code bằng tín hiệu riêng. Chỉ giữ duy nhất lớp vàng (evidence) để hình nhìn gọn: vàng chính là chỗ model dựa vào khi cho điểm cao/thấp.

Minh họa 2 cặp code trong demo

Cặp 1: hai file **giống hệt** trong `data.zip` (cùng bài `spit0035`) — cả 5 model clone. Cặp 2: hai bài **khác nhau** của POJ-104 (`test-neg-1196`, bài 30 với 33) — ví dụ ngược, vì 10 file trong `data.zip` đều cùng một bài. Điểm và ngưỡng lấy từ đúng bộ model và `threshold.json`.

Quy ước màu trong minh họa bên dưới: vàng = model đang chú ý (evidence); không màu = phần còn lại (khớp thuần hoặc khác nhau). Bỏ hẳn lớp xanh "đoạn khớp" để nhìn gọn — chỉ cần biết model tập trung vào đâu.

9.1 Cặp trùng khớp (giống hệt) — mã 355523 với 355575

Căn chỉnh: 117/117 token phía A, 117/117 phía B, 1 khối, phủ A 100,0%, B 100,0%.

Phương pháp	Điểm	Ngưỡng	Kết luận
B1 MOSS	1,000	0,010	clone
M1	1,000	0,590	clone
M2	1,000	0,430	clone
M3	0,985	0,420	clone
M4	0,962	0,420	clone

Vệt vàng theo model: B1 · MOSS: kgram, A 1 / B 1 chỗ; M1 · CodeBERT: saliency, A 26 / B 25 chỗ; M2 · GraphCodeBERT: saliency, A 30 / B 30 chỗ; M3 · ASTNN: stmt, A 13 / B 13 chỗ; M4 · CPG-GNN: gat, A 4 / B 4 chỗ.

B1 · MOSS — 1,000 (clone)

File A (355523)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 long long a,b;
 cin>>a>>b;
 if (a>b){
 cout<< "deficit" <<endl;
 cout<<a-b<<endl;
 }
 else if (a==b){
 cout << "break-even" <<endl;
 cout<<a-b;
 }
 else if (a<b){
 cout<< "profit" <<endl;
 cout<<b-a;
 }
 return 0;
}
```

File B (355575)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 long long a,b;
 cin>>a>>b;
 if (a>b){
 cout<< "deficit" <<endl;
 cout<<a-b<<endl;
 }
 else if (a==b){
 cout << "break-even" <<endl;
 cout<<a-b;
 }
 else if (a<b){
 cout<< "profit" <<endl;
 cout<<b-a;
 }
 return 0;
}
```

M1 · CodeBERT — 1,000 (clone)

File A (355523)

```
#include <bits/stdc++.h>
using namespace std ;
int main() {
long long a, b ;
cin>>a>> b;
if (a>b){
cout<< "deficit" << endl;
cout << a-b<<endl;
}
else if (a==b){
cout << "break-even" <<endl ;
cout<<a-b;
}
else if (a<b){
cout<< "profit" <<endl ;
cout<<b-a;
}
return 0;
}
```

File B (355575)

```
#include <bits/stdc++.h>
using namespace std ;
int main() {
long long a, b ;
cin>>a>>b ;
if (a>b){
cout << "deficit" << endl;
cout<<a-b<<endl;
}
else if (a==b){
cout << "break-even" <<endl ;
cout<<a-b;
}
else if (a<b){
cout << "profit" <<endl ;
cout<<b-a;
}
return 0;
}
```

M2 · GraphCodeBERT — 1,000 (clone)

File A (355523)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
long long a,b;
cin >>a>>b ;
if (a>b){
cout<< "deficit" << endl;
cout<<a-b<<endl;
}
else if ( a == b){
cout << "break-even" << endl;
cout<<a-b;
}
else if (a< b){
cout<< "profit" <<endl ;
cout<<b-a;
}
return 0;
}
```

File B (355575)

```
#include <bits/stdc++.h>
using namespace std;
int main () {
long long a,b;
cin >>a>> b;
if (a>b){
cout << "deficit" << endl;
cout<<a-b<<endl;
}
else if ( a == b){
cout << "break-even" << endl;
cout<<a-b;
}
else if ( a< b){
cout<< "profit" <<endl ;
cout<<b-a;
}
return 0;
}
```

M3 · ASTNN — 0,985 (clone)

File A (355523)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 long long a,b;
 cin>>a>>b;
 if (a>b){
 cout<< "deficit" <<endl;
 cout<<a-b<<endl;
 }
 else if (a==b){
 cout << "break-even" <<endl;
 cout<<a-b;
 }
 else if (a<b){
 cout<< "profit" <<endl;
 cout<<b-a;
 }
 return 0;
}
```

File B (355575)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
 long long a,b;
 cin>>a>>b;
 if (a>b){
 cout<< "deficit" <<endl;
 cout<<a-b<<endl;
 }
 else if (a==b){
 cout << "break-even" <<endl;
 cout<<a-b;
 }
 else if (a<b){
 cout<< "profit" <<endl;
 cout<<b-a;
 }
 return 0;
}
```

M4 · CPG-GNN — 0,962 (clone)

File A (355523)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
long long a,b;
cin>>a>>b;
if (a>b){
cout<<"deficit"<<endl;
cout<<a-b<<endl;
}
  else if (a==b){
cout <<"break-even"<<endl;
  cout<<a-b;
}
  else if (a<b){
cout<<"profit"<<endl;
cout<<b-a;
}
return 0;
}
```

File B (355575)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
long long a,b;
cin>>a>>b;
if (a>b){
cout<<"deficit"<<endl;
  cout<<a-b<<endl;
}
else if (a==b){
cout <<"break-even"<<endl;
  cout<<a-b;
}
  else if (a<b){
cout<<"profit"<<endl;
cout<<b-a;
}
return 0;
}
```

9.2 Cặp không trùng khớp (POJ-104, hai bài toán khác nhau) — mã poj104-train-14763 với poj104-train-16014

Căn chỉnh: 7/95 token phía A, 7/189 phía B, 1 khối, phủ A 7,4%, B 3,7%.

Phương pháp	Điểm	Ngưỡng	Kết luận
B1 MOSS	0,000	0,010	không clone
M1	0,000	0,590	không clone
M2	0,002	0,430	không clone
M3	0,178	0,420	không clone
M4	0,294	0,420	không clone

Vệt vàng theo model: B1 · MOSS: kgram, A 0 / B 0 chỗ; M1 · CodeBERT: saliency, A 28 / B 47 chỗ; M2 · GraphCodeBERT: saliency, A 21 / B 57 chỗ; M3 · ASTNN: stmt, A 7 / B 7 chỗ; M4 · CPG-GNN: gat, A 6 / B 4 chỗ.

B1 · MOSS — 0,000 (không clone)

File A (bài 30 · 14763)

```
int main(){
int n,sum=0;
cin>>n;
for (int i=1;i<=n;i++)
{
if(!(i%10==7)&&!(i%7==0)&&!(i/10==7)){
sum=sum+i*i;}
}
cout<<sum<<endl;
return 0;
}
```

File B (bài 33 · 16014)

```
int main() {
int n,i;
char a[9999][256],*p;
scanf("%d",&n);
for(i=0;i<n;i++){
scanf("%s",a[i]);

}
for(i=0;i<n;i++){
for(p=a[i];*p!='\0';p++){
if(*p=='A'){*p='T';}
else if(*p=='C'){*p='G';}
else if(*p=='G'){*p='C';}
else if(*p=='T'){*p='A';}
}
}
for(i=0;i<n;i++){
puts(a[i]);}


return 0;
}
```

M1 · CodeBERT — 0,000 (không clone)

File A (bài 30 · 14763)

```
int main() {
int n ,sum=0;
cin>>n ;
for (int i= 1 ;i <=n ;i++)
{
if(!( i%10==7)&&!(i%7== 0)&&!( i/10==7)){
sum=sum + i * i ;}
}
cout<<sum<<endl;
return 0;
}
```

File B (bài 33 · 16014)

```
int main () {
int n,i;
 char  a[9999][256],* p ;
scanf ("%d",& n);
for(i=0;i<n;i++){
scanf("%s" ,a[i]);

}
for(i=0;i<n;i++){
  for(p=a[i];*p !='\0'; p++){
if(*p ==='A'){* p ='T';}
  else if(*p ==='C'){* p ='G';}
else if(*p ==='G'){* p ='C';}
else if(*p== 'T' ){*p ='A' ;}
}
  }
for(i=0;i<n;i++){
puts (a[i]);}


return 0;
}
```

M2 · GraphCodeBERT — 0,002 (không clone)

File A (bài 30 · 14763)

```
int main() {
int n,sum=0;
cin>>n;
for (int i=1;i<=n;i++)
{
if(!(i%10==7)&&!(i%7==0)&&!(i/10==7)){
sum=sum+i*i;}
}
cout<<sum<<endl;
return 0;
}
```

File B (bài 33 · 16014)

```
int main() {
int n,i;
char a[9999][256],*p;
scanf("%d",&n);
for(i=0;i<n;i++){
scanf("%s",a[i]);

}
for(i=0;i<n;i++){
for(p=a[i];*p!='\0';p++){
 if(*p=='A'){*p='T';}
 else if(*p=='C'){*p='G';}
else if(*p=='G'){*p='C';}
else if(*p=='T'){*p='A';}
}
}
for(i=0;i<n;i++){
puts(a[i]);}


return 0;
}
```

M3 · ASTNN — 0,178 (không clone)

File A (bài 30 · 14763)

```
int main() {
 int n,sum =0;
 cin>>n;
 for (int i=1;i<=n;i++)
 {
 if(!(i%10==7)&&!(i%7==0)&&!(i/10==7)){
 sum=sum+i*i;}
 }
 cout<<sum<<endl;
 return 0;
}
```

File B (bài 33 · 16014)

```
int main()  {
 int n,i ;
 char a[9999][256],*p;
 scanf("%d",&n);
 for(i=0;i<n;i++){
 scanf("%s",a[i]);

 }
 for(i=0;i<n;i++){
 for(p=a[i];*p!='\0';p++){
 if(*p=='A'){*p='T';}
 else if(*p=='C'){*p='G';}
 else if(*p=='G'){*p='C';}
 else if(*p=='T'){*p='A';}
 }
 }
 for(i=0;i<n;i++){
 puts(a[i]);}


 return 0;
}
```

M4 · CPG-GNN — 0,294 (không clone)

File A (bài 30 · 14763)

```
int main() {
int n,sum=0;
  cin>>n;
for (int i=1;i<=n;i++)
  {
if(!(i%10==7)&&!(i%7==0)&&!(i/10==7)){
  sum=sum+i*i;}
}
  cout<<sum<<endl;
return 0;
}
```

File B (bài 33 · 16014)

```
int main() {
int n,i;
char a[9999][256],*p;
scanf("%d",&n);
for(i=0;i<n;i++){
scanf("%s",a[i]);

}
for(i=0;i<n;i++){
for(p=a[i];*p!='\0';p++){
if(*p=='A'){*p='T';}
else if(*p=='C'){*p='G';}
else if(*p=='G'){*p='C';}
else if(*p=='T'){*p='A';}
}
}
  for(i=0;i<n;i++){
puts(a[i]);}


return 0;
}
```

Hạn chế

- Mỗi phương pháp chạy một lần với seed 42.
- ASTNN dừng ở epoch 7 (early stopping, tốt nhất epoch 4).
- CPG-GNN dừng ở epoch 5 (hết epoch, val F1 còn tăng) — có thể train thêm.
- CodeBERT, GraphCodeBERT cắt hàm còn 256 token; 78,6% hàm trong tập train bị cắt.
- Demo data.zip chỉ 10 file cùng một bài: minh họa trực quan, không thay kết quả test 11.000 cặp ở trên.

Kế hoạch tiếp theo

- Train thêm CPG-GNN vài epoch (val F1 còn tăng) và chạy vài seed.
- Chạy lại CodeBERT, GraphCodeBERT vài seed; thử độ dài đầu vào 512 token.
- Đưa highlight evidence vào báo cáo tự động của quét zip (hiện chỉ xem tay trong modal).